



ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
HAROKOPIO UNIVERSITY

Matrix Factorization

Τεχνολογίες Γραφημάτων

Νίκος Γουρνακης, it22023

03 Αυγουςτου 2024

1)

Η υλοποίηση το αλγορίθμου, όπως είναι στην δημοσίευση που μας δώσατε, είναι στο αρχείο **StochasticGradientDescent.py**. Βεβαία εβαλα κάποια εξτρα υλοποίηση βελτιστων αποτελεσμάτων το οποίο είναι στο αρχείο `utils.py`, το function `find_best_threshold_rank`.

Γιατι χρειαζομαστε αυτην την εξτρα συναρτηση;

Ας τυπωσουμε το συνθετικο adjacency matrix

```
import networkx as nx
import StochasticGradientDescent

G = nx.DiGraph([(0, 1), (1, 2), (0, 2), (2, 3), (3, 0), (3, 4)])
print(nx.adjacency_matrix(G).todense())
Z = StochasticGradientDescent.sequential_stochastic_gradient_descent(G, r=2)
Y = Z @ Z.T
print(Y)
```

```
[[0 1 1 0 0]
 [0 0 1 0 0]
 [0 0 0 1 0]
 [1 0 0 0 1]
 [0 0 0 0 0]]
[[1.01043248 1.04423791 0.94406747 1.04846002 0.06768637]
 [1.04423791 1.08082466 0.97841599 1.09361521 0.07119044]
 [0.94406747 0.97841599 0.88668861 0.996472 0.06531632]
 [1.04846002 1.09361521 0.996472 1.14945625 0.07780287]
 [0.06768637 0.07119044 0.06531632 0.07780287 0.00546515]]
```

Ο πρωτος πινακας είναι το πραγματικο adjacency matrix του γραφου και ο δευτερος είναι ο συνθετικος. Βλεπουμε οτι ο δευτερος δεν είναι μονο ασσοι και μηδενικα, αρα πρεπει να βαλουμε εναν οριο οπου πανω απο το οριο η τιμη είναι ασσος και κατω απο το οριο η τιμη είναι μηδεν.

Εδω έχουμε κομμάτι της συνάρτησης `find_best_threshold_rank` που τα comments περιγράφουν την διαδικασία για να βρούμε το βελτιστο οριο:

```
Z = sequential_stochastic_gradient_descent(G, rank)
synthetic_adjacency_matrix = Z @ Z.T

# We will iterate over a range of thresholds to find the best one
min_val = np.min(synthetic_adjacency_matrix)
max_val = np.max(synthetic_adjacency_matrix)
for threshold in np.arange(min_val, max_val, 0.01):
    print(f"Testing rank: {rank}, threshold: {threshold}")
    # we need to apply a threshold to the synthetic adjacency matrix, because the original graph has 0s and 1s
    # and the synthetic matrix has floats. By applying a threshold, we can "interpret" the synthetic matrix as a
    # matrix of 0s and 1s
    synthetic_adjacency_matrix_thresholded = np.where(synthetic_adjacency_matrix > threshold, 1, 0)

    # Finding the precision of the factorized matrix
    nodes_that_match = nx.to_numpy_array(G) == synthetic_adjacency_matrix_thresholded
    precision = np.sum(nodes_that_match) / (G.number_of_nodes() * G.number_of_nodes())

    if precision > best_precision:
        best_precision = precision
        best_threshold = threshold
```

Οπου ετσι, δεν θετουμε τυχαια ενα νουμερο σαν οριο, αλλα βρισκουμε το οριο οπου εχουμε βελτιστη ομοιοτητα.

Ο παρακατω κωδικας βρισκεται στο αρχειο `testing.py`

```
best_threshold, _, best_precision = find_best_threshold_rank(G, starting_rank: 2)
Y_with_threshold = np.where(Y > best_threshold, 1, 0)
print(Y_with_threshold)
print(f"{best_precision * 100}%")
```

Και εδω βλεπουμε τον συνθετικο πινακα αφου εφαρμόσουμε το οριο:

```
[[1 1 0 1 0]
 [1 1 0 1 0]
 [0 0 0 0 0]
 [1 1 0 1 0]
 [0 0 0 0 0]]
72.0%
```

Εχουμε ομοιοτητα 72%

Βεβαια η συνάρτηση αυτή δεν βρισκει μονο το βελτιστο οριο αλλα βρισκει και το βελτιστο rank για τον καθε γραφο, καθως ουτε και αυτο πρεπει να το βαλουμε τυχαια. Ξερούμε οτι

πρέπει να είναι ένα νούμερο πολύ μικρότερο από το N , εγώ έβαλα να ψαχνει με customizable βήμα μέχρι έως και $N/2$.

Τα συνολα δεδομένων που χρησιμοποιήθηκαν για την δοκιμή είναι τα παρακάτω

- <https://snap.stanford.edu/data/amazon0302.html>
- <https://snap.stanford.edu/data/amazon0312.html>
- <https://snap.stanford.edu/data/amazon0505.html>
- <https://snap.stanford.edu/data/amazon0601.html>
- <https://snap.stanford.edu/data/email-EuAll.html>

Αυτοί οι γραφοί είναι σε μορφή αρχείου .txt οπότε εφτιαξα μια συνάρτηση για να κάνει parse τους γραφούς σε έναν networkx γράφο. Αυτό βρίσκεται στο αρχείο `parsing.py`.

Όλα τα συνολα είναι παρα πολύ μεγάλα για δοκιμή οπότε τα περιορήσα στους 1000 κομβούς το καθένα. Παραπάνω από αυτό παίρνει υπερβολική πολύ ώρα για να βρεθεί το Z .

Αποτέλεσμα ομοιότητας:

```
The precision of file "Amazon0302.txt" is: 99.6284%. Best threshold: 3.4404806617485755. Best rank: 50
The precision of file "Amazon0312.txt" is: 99.6454%. Best threshold: 3.8707680798404542. Best rank: 50
The precision of file "Amazon0505.txt" is: 99.4225%. Best threshold: 4.902944715403474. Best rank: 50
The precision of file "Amazon0601.txt" is: 99.2953%. Best threshold: 2.3411131772818043. Best rank: 50
The precision of file "Email-EuAll.txt" is: 99.0239%. Best threshold: 1.3668671493684796. Best rank: 300
```

Ο κώδικας για τα παραπάνω αποτελέσματα είναι στο αρχείο `all_graphs_testing.py`

2)

Όπως βλέπουμε στα παραπάνω αποτελέσματα ομοιότητας, έχουμε πολύ καλή ομοιότητα, αλλά όχι 100%. Αυτό μπορούμε να το εκμεταλλευτούμε. Αμα βάλουμε ένα check ότι ο συνθετικός πίνακας θα έχει **πάντα** παραπάνω/ή ίσες ακμές με τον πραγματικό τότε μπορούμε να εκμεταλλευτούμε αυτές τις εξτρα ακμές, σαν πχ συστάσεις σε χρήστες για αγορές προϊόντων. Τα παραπάνω datasets της Amazon, κάθε κομβός είναι ένα προϊόν και μια ακμή έχει την έννοια ότι τα προϊόντα αγοράστηκαν μαζί. Αρα στα παραπάνω dataset μπορούμε να κάνουμε προτάσεις σε χρήστες, γνωρίζοντας ένα ή παραπάνω προϊόντα του καταθίου τους, τι **επίσης** να αγοράσουν που “συμπληρώνει” τα υπόλοιπα τους προϊόντα.

Εχω φτιαξει μια δεύτερη συνάρτηση για αυτόν το σκοπό `find_best_threshold_rank_amazon` όπου είναι η ίδια με την προηγούμενη `find_best_threshold_rank` απλά βάζω και αυτό το εξτρα check που αναφέρω στην παράγραφο παραπάνω.

```
if np.count_nonzero(synthetic_adjacency_matrix_with_thresholded == 1) <= 6.size():
    continue
```

στο αρχείο `amazon.py` μπορούμε να δούμε την διαδικασία για την δημιουργία συστάσεων.

αρχικά φορτώνουμε το dataset

```

1  G = parsing.amazon_parse_edges( file_name: "Amazon0601.txt", limit_nodes=1000)
2  # nx.draw(G, with_labels=True)
3  # plt.show()
4  print(f"Number of nodes: {G.number_of_nodes()}")
5  print(f"Number of edges: {G.size()}")
6

```

μετα δημιουργουμε τον συνθετικο πινακα

```

2  best_threshold, best_rank = find_best_threshold_rank_amazon(G, starting_rank=50, rank_step=50)
3  Z = sequential_stochastic_gradient_descent(G, best_rank, eps=0.001)
4  synthetic_adjacency_matrix = Z @ Z.T
5  synthetic_adjacency_matrix_with_threshold = np.where(synthetic_adjacency_matrix > best_threshold, 1, 0)
6

```

τελος ειναι το κομματι των συστασεων

```

12  edges_that_dont_match = nx.to_numpy_array(G) != synthetic_adjacency_matrix_with_threshold
11  print(edges_that_dont_match)
10  num = 0
9   num_of_positives = 0
8   coords_of_recommendations = []
7   for i, row in enumerate(edges_that_dont_match):
6       for j, col in enumerate(row):
5           if col:
4               num += 1
3               if i != j and synthetic_adjacency_matrix_with_threshold[i][j] == 1:
2                   num_of_positives += 1
1                   coords_of_recommendations.append([i, j])

```

βρισκουμε ολες τις ακμες που δεν υπαρχουν στον αρχικο γραφο, και ελεγχουμε αμα ειναι ασσος (δηλαδη συσταση) για να το αποθηκευσουμε για χρηση μετα. Δεν κοιταμε στην διαγωνιο του πινακα καθως το να προτεινουμε καποιος να αγορασει το ιδιο προιον δυο φορες δεν ειναι καλη συσταση.

Ξεκινουμε με 7046 ακμες:

```

Number of nodes: 1000
Number of edges: 7046

```

Βρισκουμε την καλυτερη ταξη και οριο για τον συνθετικο πινακα (βαση ομοιοτητας):

```

The best threshold is: 1.1393658660824646
The best rank is: 150
The precision of the factorized matrix is: 98.5765%

```

Και εχουμε τα αποτελεσματα:

```
number of edges that dont match: 13792
number of edges that dont match that are also one: 6649
```

Βλεπουμε οτι εχουμε 6649 νεες προτασεις που μπορουμε να κανουμε σε χρηστες.

Το επομενο βημα θα ηταν, αμα ειχαμε καποιο dataset απου οι ακμες να ειχαν βαρη για να μπορουμε να βαλουμε τις προτάσεις μας σε μια ιεραρχια, ετσι ωστε να εχουμε μια καλυτερη ιδεα την να προτεινουμε στον χρηστη.

Ενας τροπος για να βαλουμε βαρη στις συστασεις μας ειναι να αξιοποιήσουμε το γεγονος οτι ο συνθετικο πινακας δεν εχει μονο ασσους και μηδενικα, αλλα διαφορες τιμες. Τι μπορουμε να κανουμε ειναι να εχουμε σαν refrence το `best_threshold` που βρισκε η συναρτηση `find_best_threshold_rank_amazon` για το αμα μια τιμη αντιπροσωπεύει ασσο ή μηδεν και υστερα να εχουμε την διαφορα μεταξυ αυτου το `threshold` και της πραγματικης τιμης και να το θέσουμε αυτο σαν βαρος. Οσο πιο μεγαλη ειναι η διαφορα, τοσο πιο μεγαλο βαρος εχει.

Αυτο τον τροπο σκεψης τον υλοποιο στο αρχειο **amazon_with_weights.py** οπου ειναι στην ουσια το ιδιο script με το `amazon.py` αλλα με καποιες αλλαγες για να αποθηκευσουμε και τα βαρη.

Η πρωτη αλλαγη ειναι:

```
5
4     best_threshold, best_rank = find_best_threshold_rank_amazon(G, starting_rank=50, rank_step=50)
3     Z = sequential_stochastic_gradient_descent(G, best_rank, eps=0.001)
2     synthetic_adjacency_matrix = Z @ Z.T
1     synthetic_adjacency_matrix_with_threshold = np.where(synthetic_adjacency_matrix > best_threshold, 1, 0)
18    synthetic_adjacency_matrix_with_threshold_removed = synthetic_adjacency_matrix - best_threshold
```

στη γραμμη 18 δημιουργώ μια εξτρα μεταβλητη που αποθηκευω ολες τις τιμες του συνθετικου πινακα (πριν την εφαρμογη του οριου) πλην το οριο, ετσι ωστε ολες η αρνητικες τιμες ερμηνευονται σαν απουσια ακμης και ολες οι μη αρνητικες ερμηνευονται σαν παρουςια ακμης + βαρος.

Η δευτερη αλλαγη ειναι:

```
9     edges_that_dont_match = nx.to_numpy_array(G) != synthetic_adjacency_matrix_with_threshold
8     print(edges_that_dont_match)
7     num = 0
6     num_of_positives = 0
5     coords_of_recommendations = []
4     for i, row in enumerate(edges_that_dont_match):
3         for j, col in enumerate(row):
2             if col:
1                 num += 1
29            if i != j and synthetic_adjacency_matrix_with_threshold_removed[i][j] >= 0:
1                num_of_positives += 1
2                coords_of_recommendations.append([i, j], synthetic_adjacency_matrix_with_threshold_removed[i][j])
3
```

στην γραμμή 29 όπου πλέον ελέγχουμε την καινούρια μεταβλητή για την παρουσία ακμής, υπάρχει ακμή αμα η τιμή είναι μη αρνητική. και στην γραμμή 31 όπου αποθηκεύουμε και το βάρος στο `coords_of_recommendations`.

Τέλος:

```
6 coords_of_recommendations = sorted(coords_of_recommendations, key=lambda x: x[1], reverse=True)
7 with open("amazon_recommendations.txt", "w") as f:
8     for coords, weight in coords_of_recommendations:
9         f.write(f"{coords[0]}\t{coords[1]}\t{weight}\n")
```

Κανουμε sort βάση το βάρος, και τα αποθηκεύουμε σε ένα αρχείο με το ακόλουθο αποτέλεσμα:

19	917	933	0.9227638087295258
18	933	917	0.9227638087295258
17	914	917	0.7301375570262589
16	917	914	0.7301375570262589
15	917	953	0.6627368317589406
14	953	917	0.6627368317589406
13	917	948	0.6487380784069343
12	948	917	0.6487380784069343
11	700	917	0.6420912231506737
10	917	700	0.6420912231506737
9	917	941	0.6067182894619725
8	941	917	0.6067182894619725
7	917	951	0.5743761204905344

Η ερμηνεία αυτού του αρχείου είναι απλή, κάθε γραμμή είναι μια συστάση, το πρώτο νούμερο είναι το id του προϊόντος που έχει ο χρήστης στο καλάθι, το δεύτερο νούμερο είναι το id του προϊόντος που συστήνουμε στον χρήστη να προσθέσει στο καλάθι, και το τελευταίο νούμερο είναι το βάρος (ποσο πολύ πιστεύουμε ότι ο χρήστης πρέπει να το πάρει αυτό το δεύτερο προϊόν, όσο πιο μεγάλο νούμερο τόσο πιο πολύ πιστεύουμε ότι πρέπει να το πάρει).